CipherGuard Shield: Authenticated Encryption and Machine-Learning Intrusion Detection for Secure Cloud Data Transmission

AVUTI SANDHYA
Roll No. 23K91A0520
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

BANOTHU INDU
Roll No. 23K91A0530
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

DASARI SRITEJA
Roll No. 23K91A0559
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

BEJUGAM VAISHNAVI
Roll No. 23K91A0533
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

CH. PRABHAKAR
[DESIGNATION] (project guide)
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

***Abstract*---**Data sent to a public cloud can be read or altered in transit, and encryption alone cannot see an attacker who already holds valid access. This paper presents CipherGuard Shield, a platform that places two protections around one cloud data path. SecureChannel seals each payload with AES-256-GCM under a key derived from a fresh ECDH P-256 exchange and HKDF-SHA256. ThreatSense Engine classifies network-flow records with XGBoost and a one-dimensional CNN combined by logistic-regression stacking. Both were measured on one machine. All 5,000 of 5,000 tampered messages were rejected, one protected 100 KB transfer costs 5.7 ms of which detection is 81 %, and on the official UNSW-NB15 split the stacked detector reaches 0.890 accuracy, below XGBoost alone at 0.910. The bundled demo model, trained on synthetic data, scores 0.973 on its own test split but 0.660 on real flows, which is the main caution of the work. Under a protocol in the style of the base paper the stacked model reaches 0.938.

***Index Terms*---**authenticated encryption, cloud security, elliptic-curve Diffie-Hellman, ensemble learning, intrusion detection, stacking, UNSW-NB15.

I. INTRODUCTION

Moving data to a public cloud moves it outside the owner's network perimeter. Confidentiality, integrity and availability must then hold while the data is in transit, and operators need to notice attacks that encryption cannot prevent, such as a session opened with stolen credentials [2], [3]. Symmetric ciphers such as AES are fast but need a safe way to agree on a key; asymmetric schemes solve key agreement but are too slow for bulk data, so practical systems wrap a fresh symmetric key with an asymmetric one [4], [5], [6].

In most deployments cryptography and intrusion detection are separate products with separate operators. The surveyed cloud-security work likewise treats them separately: each proposes one layer and leaves the cost of running both unmeasured [2], [3], [7], [8]. The base paper [1] is a notable exception. It couples a quantum-inspired network and LeNet with an enhanced elliptic-curve cipher and reports a mean accuracy of 0.949 on 9,000 UNSW-NB15 flows, but it offers no formal security proof for its cipher and does not measure the combined overhead of detection plus encryption on one data path.

Done separately, the two layers also fail separately. Encryption gives no signal when valid credentials are stolen, and a flow-level detector sees only traffic statistics, not the payload that encryption hides. Running both means two sets of keys, logs and latencies, and without a combined measurement an operator cannot say whether detection or encryption dominates the cost of one transfer. The batch objectives for this work ask for a hybrid AES and RSA scheme, an intrusion classifier trained on a benchmark dataset, flagging and logging of suspicious sessions, and an evaluation of encryption time, overhead and detection accuracy.

CipherGuard Shield is a working, tested platform that makes this combination concrete at student-project scale. The contributions are: (1) SecureChannel, an authenticated-encryption path built from standard primitives (ECDH P-256, HKDF-SHA256, AES-256-GCM) with a fresh key per message, benchmarked against an RSA-OAEP hybrid; (2) ThreatSense Engine, an XGBoost and 1D-CNN detector with logistic-regression stacking, an alert log and a live console; (3) a measured evaluation on the real UNSW-NB15 partition, on a protocol like the base paper's and across synthetic and real data, including the combined per-transfer overhead; and (4) an account of what the numbers do and do not show. Section II surveys related work, Section III describes the method, Section IV gives the results and Section V concludes.

A. System Hardware

All timings and training runs used one Windows 11 machine with an Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB of RAM and no GPU, so the neural network trained and ran on the CPU. The machine was shared with other jobs, so absolute times are indicative.

B. Software Requirements

Python 3.11.9; FastAPI 0.110 and Uvicorn for the service; the cryptography package 42.0.7; XGBoost 2.0.3 [9];

scikit-learn 1.5.0 [10] for preprocessing, logistic regression and the baselines; PyTorch [11] (2.14.1, CPU) for the CNN; and React 19 with Vite for the console.

II. LITERATURE SURVEY

Base paper. Kalyanasundaram et al. [1] normalise flows with min-max scaling, classify them with a flatten-layer feature-map quantum neural network that uses eigenvalue decomposition (simulated on classical hardware) and with LeNet, and fuse the two scores with a double-sigmoid median min-max scheme (DSMMN-SLF). Non-intrusive data are then encrypted with an enhanced elliptic-curve cryptography (EECC) variant. Experiments use 9,000 balanced UNSW-NB15 samples split 70/15/15 and averaged over 10 runs: mean accuracy 0.949, precision 0.97 and false-positive rate 0.03, ahead of re-implemented baselines such as SqueezeNet (0.935) and LeNet (0.907). EECC encrypts in 0.39 s and decrypts in 0.26 s. The authors state that a rigorous proof of EECC security under standard hardness assumptions is future work, and the subset drawn from UNSW-NB15 and the baseline settings are not fully specified. Its ablation (accuracy 0.956 for the full model at an 80 % training share, between 0.902 and 0.932 when the quantum block, the fusion rule or the normalisation is replaced or removed) supports the value of each component, but every variant is evaluated on the same balanced subset, so the ablation shows relative, not absolute, merit.

IDS combined with cryptography. Kadry et al. [12] pair an optimised quantum neural network with elliptic-curve encryption of stored files. The batch survey adds encryption with anomaly detection for storage and transmission [2], an intrusion-detection system with automated data deletion across clouds [3], federated attack detection with an improved Blowfish cipher [7] and a hybrid-encryption privacy framework [8].

Detector design. Bakro et al. [13] combine SMOTE, three feature-selection methods and a random forest for a cloud IDS and report over 98 % accuracy on UNSW-NB15 in their abstract; Mbaya et al. [14] train a BiLSTM-RNN in a blockchain-backed federated setting; Rahman et al. [15] train detectors on GAN-generated data and, as summarised in [1], reach perfect BoT-IoT scores, a reminder that synthetic data can flatter a model. Stacked generalisation [16] combines base learners through a meta-model trained on held-out predictions, and gradient-boosted trees [9] remain strong on tabular flow features. UNSW-NB15 [17] supplies 49 flow features with nine attack categories and an official training and testing partition captured at different times; its class imbalance and overlap are known weaknesses [18].

Hybrid cryptography for the cloud. Mahalle and Shahade [6] protect uploads and downloads with RSA-wrapped AES keys, the classical hybrid that the batch objectives name. The survey items repeat the pattern with variations: AES and RSA with an XOR key mask, reported at about two seconds per 2 to 12 KB input [8], and a Blowfish variant inside a federated detector [7]. Such timings depend on the implementation and the payload, so they cannot be compared across papers; the benchmark here therefore measures an RSA-OAEP hybrid on the same machine as the elliptic-curve path.

TABLE I
REPRESENTATIVE RELATED WORK

Work	Approach	Data	Result as reported	Limitation
[1]	QNN + LeNet fusion, EECC	UNSW-NB15, 9,000	Acc. 0.949; enc. 0.39 s	No security proof
[12]	Optimised QNN + ECC	Network data	Acc. 98.5 % (as cited in [1])	Storage focus
[13]	SMOTE, 3 feature selectors, RF	UNSW-NB15, Kyoto	Over 98 %, over 99 %	Detection only
[14]	Blockchain federated BiLSTM	CIDDS	Precision 0.996	No crypto cost
[2]	Encryption + anomaly detection	Not stated	No figures given	Not evaluated
[3]	IDS-triggered data deletion	None	Concept only	Not evaluated
[8]	AES + RSA/MECC, XOR key mask	2-12 KB inputs	Enc. 2.09 s, dec. 2.45 s	No detector
[7]	Federated IDS, improved Blowfish	Cloud	Abstract not retrievable	Not verified

Evaluation practice. Reported accuracy on UNSW-NB15 depends heavily on how the data are split. The official partition separates training and testing in time, whereas a random subset of the training set shares its capture conditions with the training data and is easier; the base paper uses a balanced random subset of 9,000 flows. Both protocols are used here and reported separately so that the difference is visible rather than hidden in a single headline number. The visual analysis in [18] finds class imbalance and overlap to be the main problems of the dataset, which is why false-positive rate and per-category rates are reported next to accuracy.

Gap. The surveyed work builds either a detector or a cipher, or couples them without reporting what the pair costs. None reports, on one path, the encryption time, the detection time and their share of a transfer, and none tests how a detector trained on generated data behaves on real traffic. CipherGuard Shield addresses these two points with standard, auditable primitives and does not attempt the base paper's quantum-inspired network or its custom cipher.

III. METHODOLOGY

Architecture. Fig. 1 shows the layers. A React console talks to a FastAPI service on the same host. The service exposes nine endpoints (scoring, batch scoring, alerts, encrypt, decrypt, file encryption, metrics and health). Alerts and session keys live in process memory, the models and reports are files, and the cloud is a mock endpoint with its own key pair in the same process.

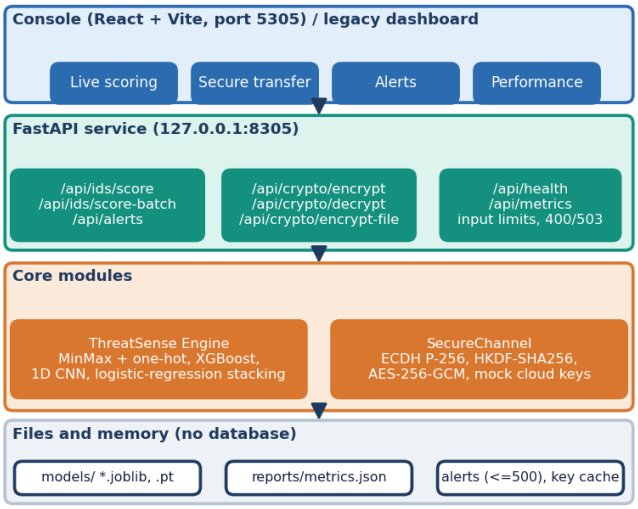


Fig. 1. CipherGuard Shield architecture: console, FastAPI service, the two core modules and file or memory storage.

Console. The React console has four working screens: live scoring of one record or a CSV batch with a session log, secure transfer (encrypt text or a file, decrypt at the mock cloud, tamper with the ciphertext and watch the tag check fail), an alert table with severity filter, and a model-performance screen that reads the held-out metrics file. A banner on that screen states which data the bundle was trained on, so a viewer sees whether the numbers come from synthetic or real flows.

SecureChannel. For every message the sender generates an ephemeral P-256 key pair (e, E), computes the ECDH shared secret with the cloud's public key C and derives a 256-bit key [5], [19]:

$$K = HKDF\text{-}SHA256(\, ECDH(e, C), info\,). \qquad (1)$$

A random 12-byte nonce n is drawn and the payload m is sealed with AES-256-GCM [4], giving a ciphertext of the same length and a 16-byte tag:

$$(c, t) = AES\text{-}GCM_K(\, n, m\,). \qquad (2)$$

The envelope (c, n, t, E) adds a fixed 93 bytes. The cloud recomputes K from its private key and E, and releases plaintext only if the tag verifies. Payloads are limited to 1 MB of text or 5 MB per file. As a baseline, an RSA-2048 and an RSA-3072 OAEP key-wrapping hybrid [20] with the same AES-GCM was benchmarked but is not part of the product.

Security properties. Because every message uses a fresh ephemeral key, a leaked session key exposes one message only, and a repeated random nonce could not matter since no key is reused. GCM authenticates the ciphertext, nonce and tag together, so any modification is detected before plaintext is released. Two limits are deliberate to state: the ephemeral public key and the cloud's public key are not signed or certified in this prototype, so an active attacker who could substitute keys is outside the threat model; and the mock cloud's private key is held in process memory and is lost on restart. Neither confidentiality of the payload against a passive eavesdropper nor tamper detection depends on these limits.

ThreatSense Engine. A flow record has 42 UNSW-NB15 features (39 numeric, three categorical). Numeric features are min-max scaled and protocol, service and state are one-hot encoded (70 inputs for the synthetic bundle, 192 for real data). XGBoost [9] (200 trees, depth 6, learning rate 0.08) and a CNN (two Conv1d layers of 32 and 64 filters with max pooling, a 64-unit dense layer, dropout 0.3 and a sigmoid output, trained with Adam and early stopping) each give an attack probability. A logistic-regression meta-model fitted on a validation split fuses them [16]:

$$p = \sigma(\, w_1\, p_{xgb} + w_2\, p_{cnn} + b\,). \qquad (3)$$

A flow is labelled attack if p reaches 0.5 and receives severity critical at 0.85 or above, high at 0.70 and medium otherwise; attacks are appended to a 500-entry alert log. The two learners were chosen because they are complementary on tabular data: trees cope with mixed scales and categorical counts, the CNN reads the feature vector as a sequence.

Features. The 42 inputs describe a flow rather than packet payloads: duration and protocol, service and connection state; packet and byte counts in each direction; rates, load, loss, inter-packet times and jitter; time-to-live values and TCP window sizes and round-trip timings; and connection-count features that summarise recent flows to the same host or service. Missing fields in an API request are filled with fixed defaults, so a request that sets only a dozen fields scores a mostly default flow.

Training. The synthetic bundle is generated from class-conditional distributions (exponential, Poisson and log-normal draws for durations, packet counts and byte counts) and then 2.5 % of the labels are flipped. For the official partition the training set is split 85/15 (stratified) into fit and validation parts; the validation part drives CNN early stopping (patience 5, at most 15 epochs, batch 256, Adam at 1e-3, unweighted binary cross-entropy) and fits the meta-model, so the test set never influences training. XGBoost uses scale_pos_weight equal to the negative-to-positive ratio. The CNN has 76,097 parameters at 70 inputs. Seeds 42 to 46 change the split and the initialisation. On the CPU, XGBoost trained in 4.4 s and the CNN in 246 s on 149,039 flows.

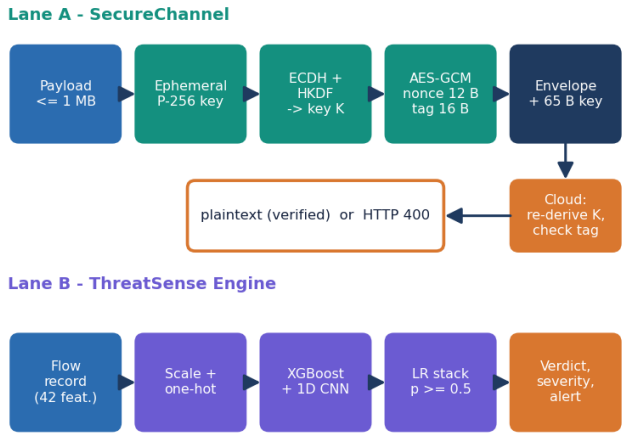


Fig. 2. Main flow: the two lanes run side by side; the API does not gate a transfer on the detector's verdict.

Data and protocols. The shipped default model is trained on 12,000 synthetic rows with the UNSW-NB15 schema (70/15/15 split, 2.5 % label noise). Because that is not real traffic, three further experiments use the real data [17]: the official partition (149,039 fit and 26,302 validation flows from the training set, 82,332 test flows, five seeds); a protocol in the style of the base paper (9,000 balanced flows, 70/15/15, ten repetitions); and a shift test that scores the shipped model on real flows and a real-trained model on the synthetic CSV. Random forest, decision tree and logistic-regression baselines use the same inputs.

Metrics. Accuracy, precision, recall, F1, ROC-AUC, false-positive rate (false alarms among normal flows) and the Matthews correlation coefficient are computed on held-out flows with attack as the positive class. Because UNSW-NB15 is imbalanced and its official split is temporal, the false-positive rate and the per-category rates matter as much as accuracy. The threshold sweep is run on the test set and is descriptive, not a tuned result.

Implementation and verification. The service validates every input (size limits, CSV and base64 checks) and returns 400 or 503 with a message. 26 pytest tests and 15 smoke checks pass against the running API. Crypto latency is the median of 15 to 300 repetitions per payload size; tamper cases use 1,000 trials each.

IV. RESULTS

Test conditions. Detection results come from the official UNSW-NB15 testing set (82,332 flows, 55.1% attacks, captured after the training set) and from 1,350-flow test splits of the base-paper-style protocol. Threshold 0.5 is used unless stated. Table II gives the main metrics; the official-partition accuracy is a mean over five seeds. Table III compares with the base paper and Table IV lists the encryption and decryption times.

TABLE II
DETECTION METRICS (MEAN OF SEEDS OR RUNS)

Setting, model	Accuracy	Prec.	Recall	F1	AUC
Official, stacked	0.890 ± 0.002	0.844	0.981	0.907	0.984
Official, XGBoost	0.910 ± 0.001	0.883	0.964	0.922	0.985
Official, 1D CNN	0.841 ± 0.004	0.780	0.991	0.873	0.976
Paper prot., stacked	0.938 ± 0.007	0.934	0.944	0.939	0.988
Paper prot., XGBoost	0.939 ± 0.007	0.938	0.939	0.939	0.989
Paper prot., RF	0.939 ± 0.008	0.938	0.941	0.939	0.989
Shipped, synthetic test	0.973	0.977	0.970	0.973	0.970
Shipped, real official test	0.660	0.691	0.692	0.692	0.657

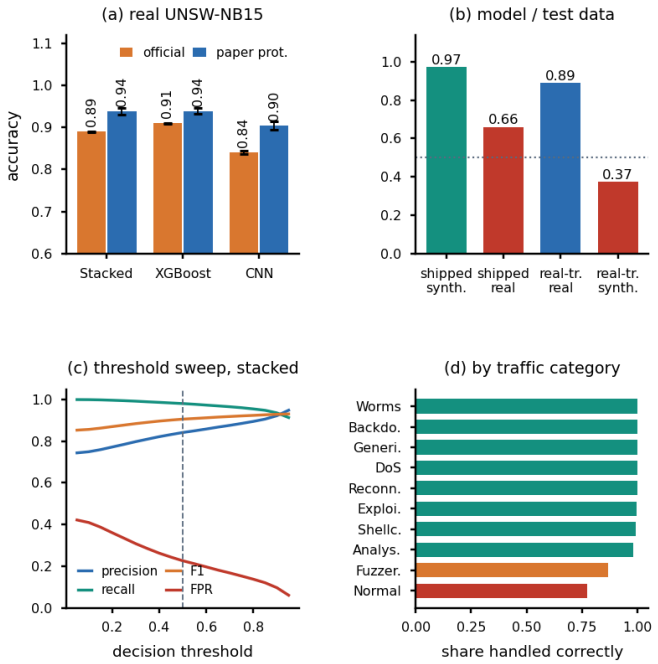


Fig. 3. (a) Accuracy on real UNSW-NB15 (mean, bar = std). (b) Stacked model by training and test data (dotted = chance). (c) Threshold sweep, official test set, seed 42. (d) Share of each traffic category handled correctly, seed 42.

Detection. On the official partition the stacked ensemble reaches 0.890 ± 0.002 accuracy with ROC-AUC 0.984, but XGBoost alone is better (0.910 ± 0.001, false-positive rate 0.156 against 0.222). The CNN is weakest (0.841); the meta-model is fitted on a validation split from the earlier training period, so it inherits the CNN's higher false-alarm rate on the later test period. Seed-to-seed spread is small (standard deviation 0.0016 for the stack, 0.0009 for XGBoost and 0.0041 for the CNN), and the Matthews coefficient is 0.786 for the stack and 0.820 for XGBoost, so the ranking of the models is not an artefact of one split. The fitted meta-model weights XGBoost (6.77) above the CNN (2.97), consistent with the ranking of the two. Averaging the two probabilities gives 0.888, so stacking adds only about 0.002. Under the paper-style protocol stacking, XGBoost and a random forest are indistinguishable (0.938, 0.939, 0.939).

Observed limits. At threshold 0.5 the stacked model passes only 77.4% of normal flows (8,354 false alarms against 877 missed attacks, seed 42), so an operator would see about 23 alarms per 100 normal flows. Raising the threshold to 0.9 gives accuracy 0.921, precision 0.922, recall 0.936 and false-positive rate 0.096. Fuzzers are the weakest attack category (13.3 % missed); every other category is detected at 98 % or more. The source time-to-live field sttl dominates XGBoost's feature gain, a known artefact of the UNSW-NB15 testbed, so the scores may overstate how well the model would generalise to other networks.

Cases that went wrong. On the console's attack preset the two learners disagree strongly: XGBoost gives an attack probability of 0.45 and the CNN 1.00, and only the fused value of 0.84 crosses the threshold, a reminder that a single learner can miss what the other sees. The shipped bundle

labels real traffic unreliably (the real sample in Fig. 5 is 65 % agreement), unseen protocol, service or state values are ignored by the encoder instead of rejected, and a record with default values for most fields can look normal or abnormal by accident. Malformed CSV files, files over the size limit, empty payloads, bad base64 and tag mismatches are rejected with HTTP 400, and an unavailable model returns 503; these paths are covered by the tests.

Comparison with the base paper. Table III sets the figures side by side. The base paper reports a mean accuracy of 0.949, precision 0.97 and false-positive rate 0.03 for its model, with SqueezeNet the strongest baseline at 0.935. Under a comparable protocol our stacked model reaches 0.938 accuracy, 0.934 precision and 0.067 false-positive rate: about one point below the proposed model and level with its strongest baseline. The comparison is indicative only. The paper's sampled subset, hyper-parameters and test fraction are not published, our test splits hold 1,350 flows, and its quantum-inspired network and EECC were not reproduced.

TABLE III
COMPARISON WITH THE BASE PAPER (INDICATIVE)

Model	Accuracy	Precision	FPR
Base paper, proposed [1]	0.949	0.97	0.03
Base paper, best baseline	0.935	n/a	n/a
Ours, stacked (paper protocol)	0.938 ± 0.007	0.934	0.067
Ours, XGBoost (paper protocol)	0.939 ± 0.007	0.938	0.062
Ours, stacked (official split)	0.890 ± 0.002	0.844	0.222

Synthetic against real data. The shipped model scores 0.973 on its synthetic test split, 0.975 on freshly generated synthetic data, but 0.660 accuracy (ROC-AUC 0.657) on the 82,332 real test flows; a real-trained model scores 0.373 on the synthetic CSV. The 2.5 % label noise caps accuracy on the synthetic data at about 0.975, and XGBoost, the CNN and the stack all score the same there, so that figure measures the generator, not the detector. In the console, 120 real flows are agreed with by the shipped model in 78 cases (65.0 %) and by the real-trained bundle in 101 (84.2 %) (Fig. 5).

TABLE IV
MEDIAN ENCRYPT / DECRYPT TIME IN MS

Payload	SecureChannel	AES-GCM only	RSA-2048 hybrid	RSA-3072 hybrid
1 KB	0.12 / 0.10	0.01 / 0.01	0.03 / 0.45	0.04 / 1.35
100 KB	0.51 / 0.30	0.03 / 0.03	0.05 / 0.47	0.07 / 1.44
1 MB	4.26 / 2.99	0.79 / 0.80	0.81 / 1.23	0.97 / 2.24
5 MB	21.39 / 14.21	3.89 / 3.33	3.77 / 3.69	3.81 / 4.70

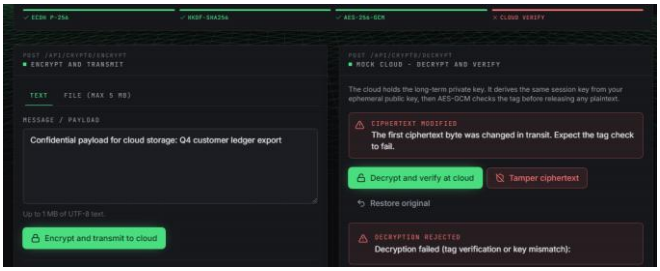


Fig. 4. Secure transfer screen: the first ciphertext byte was changed in transit and the cloud rejects the message (tag verification failed).

Cryptography. 5,000 of 5,000 tampered envelopes (flipped ciphertext, tag or nonce bit, truncation, wrong key; 1,000 trials each) were rejected and untouched messages were always accepted. 20,000 encryptions of one plaintext produced 20,000 distinct nonces and ciphertexts. For a 1 KB payload SecureChannel takes 0.12 ms to encrypt and 0.10 ms to decrypt; the RSA-2048 hybrid encrypts faster (0.03 ms) but decrypts slower (0.45 ms, and 1.35 ms for RSA-3072), and its wrapped key is 256 bytes against a 65-byte ephemeral key. At 1 MB SecureChannel needs 4.3 ms to encrypt and 3.0 ms to decrypt, more than the raw AES-GCM cost of 0.79 ms because the API payload is base64 coded.

Combined overhead. One protected 100 KB transfer (score one flow, encrypt, decrypt) takes 5.67 ms in process, split into IDS scoring 3.92 ms (81 %), encryption 0.54 ms and decryption 0.38 ms. Through the HTTP API on 127.0.0.1 the three calls for a 1 KB payload take 8.1 ms; one scoring call alone is 5.5 ms, a 1 KB encrypt call 1.2 ms, a 120-row batch 19 ms and a 1 MB file encryption 17 ms. Detection, not encryption, dominates the cost. With the 192-input real-trained models a single flow takes 3.2 ms and 1,000 flows 43 ms, so the detector would keep up with thousands of flows per second on this machine but was not tested under concurrent load.

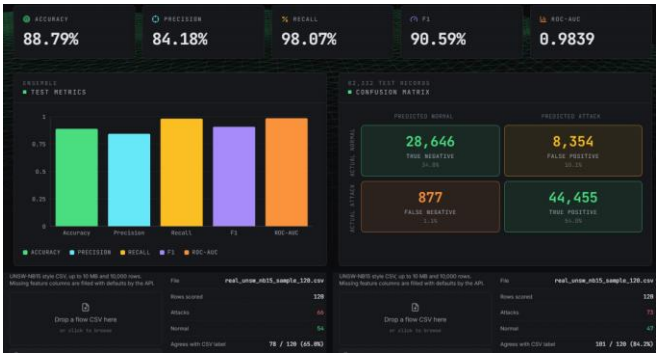


Fig. 5. Output screens with real data: held-out metrics of the real-trained bundle (top), and 120 real flows scored by the shipped model (bottom left, 65.0 % agreement) and by the real-trained bundle (bottom right, 84.2 %).

V. CONCLUSION

CipherGuard Shield delivers an authenticated-encryption path and a machine-learning intrusion detector as one tested platform with a live console. Against the batch objectives, the hybrid scheme was met by substitution (ECDH instead of RSA, with RSA benchmarked), key generation and distribution were met but key storage was not (the cloud key lives in memory), encrypt-before-upload and decrypt-after-download were met against a mock cloud, and the detector and the evaluation were met on real UNSW-NB15 data.

The key findings are that standard primitives reject every tampered message at sub-millisecond cost for payloads up to 100 KB, that detection accounts for 81 % of a protected transfer's time, and that the shipped model's 0.973 accuracy comes from synthetic data and falls to 0.660 on real flows. On real data XGBoost alone (0.910) beat the stacked ensemble (0.890). Limitations: no live packet capture, no authentication or TLS in the demo, an in-memory alert log, 23% false alarms on normal traffic at the default threshold, no adversarial testing and a test machine shared with other jobs.

Two objectives are only partly met. Suspicious flows are scored, flagged and logged with a severity, but the alert is not tied to a particular transfer and nothing is blocked automatically; and evaluation of enc/dec time, overhead and accuracy is complete for this prototype but not for a deployment, because the cloud is simulated in the same process and no network latency, concurrency or memory cost was measured.

The result most worth carrying forward is methodological: a detector must be judged on real traffic under a temporally separated split, with false alarms and per-category rates reported, and the cost of encryption must be measured next to the cost of detection on the same path. Both were done here on a single machine and a single dataset, so the findings describe this prototype and UNSW-NB15, not cloud traffic in general.

Future work: ship the real-trained model as the default and tune its threshold on a validation period; add per-category and calibrated outputs to cut false alarms; couple the verdict to the transfer so that flagged sessions are held; persist keys in a key-management service with TLS and authentication; and evaluate against evasion attacks and on newer datasets.

REFERENCES

[1] V. Kalyanasundaram, S. Raja and S. Kanthimathi, "Intrusion detection model using fused quantum NN-LeNet with improved elliptic curve cryptography based data security," *IEEE Access*, vol. 14, pp. 63767-63779, 2026, doi: 10.1109/ACCESS.2026.3686106.

[2] H. Boye, "Strengthening cloud security through advanced encryption and anomaly detection techniques for secure data storage and transmission," *FMDB Trans. Sustain. Comput. Lett.*, vol. 2, no. 3, pp. 153-163, 2024, doi: 10.69888/FTSCL.2024.000241.

[3] J. Singh and G. S. Bhathal, "An automated data deletion: a secure method for multi-cloud security with intrusion detection system," *Int. J. Sci. Res. (IJSR)*, vol. 13, no. 8, pp. 213-217, Aug. 2024, doi: 10.21275/SR24730194843.

[4] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007, doi: 10.6028/NIST.SP.800-38D.

[5] E. Barker, L. Chen, A. Roginsky, A. Vassilev and R. Davis, "Recommendation for pair-wise key-establishment schemes using discrete logarithm cryptography," NIST Special Publication 800-56A, Rev. 3, Apr. 2018, doi: 10.6028/NIST.SP.800-56Ar3.

[6] V. Mahalle and A. Shahade, "Enhancing the data security in cloud by implementing hybrid (RSA & AES) encryption algorithm," in *Proc. Int. Conf. Power, Autom. Commun. (INPAC)*, 2014, pp. 146-149, doi: 10.1109/INPAC.2014.6981152.

[7] M. B. Shinde and P. P. Ghadekar, "Federated learning-based attack detection and secure data transmission in cloud using improved Blowfish encryption," *Eng. Appl. Artif. Intell.*, vol. 182, Art. no. 115975, 2026, doi: 10.1016/j.engappai.2026.115975.

[8] N. Pamarthi and N. Rao N, "A cloud-based data privacy framework using hybrid encryption algorithms for secure data transmission," *Genet. Mol. Res.*, 2026, doi: 10.4238/mq5a9v31.

[9] T. Chen and C. Guestrin, "XGBoost: a scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, San Francisco, CA, USA, 2016, pp. 785-794, doi: 10.1145/2939672.2939785.

[10] F. Pedregosa et al., "Scikit-learn: machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825-2830, 2011.

[11] A. Paszke et al., "PyTorch: an imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, arXiv:1912.01703.

[12] H. Kadry, A. Farouk, E. A. Zanaty and O. Reyad, "Intrusion detection model using optimized quantum neural network and elliptical curve cryptography for data security," *Alexandria Eng. J.*, vol. 71, pp. 491-500, May 2023, doi: 10.1016/j.aej.2023.03.072.

[13] M. Bakro, R. R. Kumar, A. A. Alabrah, Z. Ashraf, M. N. Ahmed, M. Shameem and A. Abdelsalam, "An improved design for a cloud intrusion detection system using hybrid features selection approach with ML classifier," *IEEE Access*, vol. 11, pp. 64228-64247, 2023, doi: 10.1109/ACCESS.2023.3289405.

[14] E. B. Mbaya, E. Adetiba, J. A. Badejo, J. S. Wejin, O. Oshin and O. Isife, "SecFedIDM-V1: a secure federated intrusion detection model with blockchain and deep bidirectional long short-term memory," *IEEE Access*, vol. 11, pp. 116011-116025, 2023, doi: 10.1109/ACCESS.2023.3325992.

[15] S. Rahman, S. Pal, S. Mittal, T. Chawla and C. Karmakar, "SYN-GAN: a robust intrusion detection system using GAN-based synthetic data for IoT security," *Internet Things*, vol. 26, Art. no. 101212, Jul. 2024, doi: 10.1016/j.iot.2024.101212.

[16] D. H. Wolpert, "Stacked generalization," *Neural Netw.*, vol. 5, no. 2, pp. 241-259, 1992, doi: 10.1016/S0893-6080(05)80023-1.

[17] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, Canberra, Australia, 2015, pp. 1-6, doi: 10.1109/MilCIS.2015.7348942.

[18] Z. Zoghi and G. Serpen, "UNSW-NB15 computer security dataset: analysis through visualization," arXiv:2101.05067, 2021.

[19] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," IETF RFC 5869, May 2010, doi: 10.17487/RFC5869.

[20] K. Moriarty, B. Kaliski, J. Jonsson and A. Rusch, "PKCS #1: RSA cryptography specifications version 2.2," IETF RFC 8017, Nov. 2016, doi: 10.17487/RFC8017.